

Introduction to Machine Learning

Day 1: Detailed learning notes

A step-by-step companion to the slides and Python notebook, written for learners meeting machine learning for the first time.

Inside these notes

The ideas behind machine learning; the five learning settings; engineering applications; Python essentials; a worked learning loop; decision trees; nearest neighbours; clustering; and fully explained exercises.

How to study

Keep the Day 1 notebook open beside this document. Read a short section, run the matching cell, and explain its output in your own words. The longer explanations are here for you to return to after class. Prepared from the supplied 29-slide Day 1 deck, hands-on notebook, and speaker notes.

Numerical examples were checked by running the supplied code.

Contents

1	Getting started	4
1.1	What you are learning today	4
1.2	Where Day 1 fits in the week	4
1.3	How these notes follow the materials	4
2	What machine learning means	5
2.1	Artificial intelligence, machine learning, and deep learning	5
2.2	What changes when we learn from examples?	5
2.3	A short history and why it matters	6
2.4	Why data, computing power, and tools matter	6
2.5	The vocabulary, using a house table	7
2.6	Generalisation and overfitting	7
3	Five learning settings	8
3.1	Supervised learning	8
3.2	Unsupervised learning	8
3.3	Semi-supervised learning	9
3.4	Self-supervised learning	9
3.5	Reinforcement learning	10
3.6	A comparison you can return to	11
4	The ML workflow, from question to usable system	11
4.1	Define the problem	11
4.2	Collect suitable data	11
4.3	Explore the data and plan the split	12
4.4	Clean the data and construct features	12
4.5	Train and evaluate	12
4.6	Deploy, monitor, and improve	12
5	Where these ideas appear in practice	13
5.1	Computer vision	13
5.2	Bioinstrumentation and physiological signals	13
5.3	Robotics, finance, agriculture, language, and transport	13
5.4	TinyML and edge AI	14
6	Python, the toolbox, and your notebook	15

6.1	Why Python is used throughout the course	15
6.2	The tools and what each one contributes	15
6.3	What Google Colab is doing	16
6.4	Your first two cells	16
6.5	A small guide to the punctuation	17
7	Python essentials, explained line by line	17
7.1	Variables and types	17
7.2	Putting values into a sentence	17
7.3	Lists, indexing, and slicing	18
7.4	Dictionaries and named values	18
7.5	Loops and conditions	19
7.6	Functions and return values	19
7.7	Exercise 1: full solution and reasoning	20
8	Meeting the Iris dataset	20
8.1	What the measurements describe	20
8.2	Loading the table	21
8.3	Checking dimensions and counts	21
8.4	Reading the scatter plot	22
9	Watching a model learn, one calculation at a time	23
9.1	The six invented houses	23
9.2	Arrays and the starting settings	24
9.3	Making the first predictions	24
9.4	How the adjustment is calculated	25
9.5	Every line of the learning loop	25
9.6	Reading the plots and predicting another house	26
9.7	Changing the learning rate and starting value	27
9.8	How widely this learning loop applies	27
10	Training the first flower classifier	28
10.1	Load the inputs and targets	28
10.2	Split the flowers into training and test sets	28
10.3	Create and fit the model	29
10.4	Score the held-out examples	29
10.5	What 100 percent on this split tells us	29
10.6	Compare predictions with their true labels	30

10.7 Predict a new flower	30
10.8 Drawing and reading the tree	31
11 Nearest neighbours and the model exercises	32
11.1 How KNN makes a decision	32
11.2 The code and why it feels familiar	32
11.3 Why scale and the neighbour count matter	33
11.4 Exercise 2, task 1: change the test proportion	33
11.5 Exercise 2, task 2: vary the neighbour count	34
11.6 Exercise 2, task 3: another new flower	34
12 Finding groups with K-Means	35
12.1 What information the algorithm receives	35
12.2 The meaning of a centre	35
12.3 Fit the clustering model	35
12.4 Compare the two plots	36
12.5 What the checked grouping contains	36
13 Using ML responsibly and interpreting claims carefully	37
13.1 Representativeness and bias	37
13.2 Privacy and data handling	37
13.3 Data quality and confident mistakes	37
13.4 Responsible learning habits	38
14 Troubleshooting the notebook	38
14.1 An error is information about what happened	38
14.2 A reliable way to recover your experiment	39
15 Review, worked answers, and preparation for Day 2	39
15.1 Questions to answer in your own words	39
15.2 Worked answers	39
15.3 Questions beginners often ask	40
15.4 Before Day 2	40
16 Glossary and material guide	41
16.1 Day 1 glossary	41
16.2 Where to find each part of the supplied material	42
16.3 Sources and verification notes	42

1 Getting started

Material covered: Slides 1–3; notebook introduction; speaker-note session goals.

1.1 What you are learning today

Imagine recording the vibration of a motor every few seconds. After several weeks, you have measurements from periods when the motor was healthy and periods when it developed a fault. You would like a computer to use those examples to recognise signs of a fault in a new recording. This is the kind of problem machine learning can help you solve.

Machine learning, usually shortened to ML, is a way of building systems whose behaviour is learned from data. A learning procedure looks for useful patterns in examples. The result is a model that can produce an output for an input. For the motor, the input could be a set of vibration measurements, and the output could be a predicted operating condition.

By the end of Day 1, you should be able to explain what a model learns, recognise the main learning settings, identify inputs and targets in a table, and run a small ML experiment. You will also read the Python behind that experiment. Whenever a new term appears, connect it to something concrete: a row in a table, a number in an equation, or a line in the notebook.

You do not need to understand every mathematical detail before running the examples. Work through the calculations that are shown, ask what each number represents, and build familiarity through practice. The house-price example uses only six houses so that you can follow the whole process.

1.2 Where Day 1 fits in the week

Day	Main focus	What this adds
1	Foundations and first models	Vocabulary, learning settings, Python, and an initial working experiment.
2	Data and supervised learning	NumPy, Pandas, visualisation, exploratory data analysis, regression, and classification.
3	More models and clustering	Decision-making with further algorithms, model comparison, K-Means, and dimensionality reduction.
4	Deep learning and vision	Neural networks and learning from images.
5	Practical applications	Embedded devices, biomedical signals, robotics, reinforcement learning, and generative AI.

The three-hour Day 1 session moves from ideas to applications and then to Python. The slides allocate about 15 minutes to the welcome, 40 minutes to ML foundations, 40 minutes to learning settings, a 10-minute break, 30 minutes to applications, 35 minutes to tools and practice, and 10 minutes to the recap. These notes are longer than a spoken lesson because they also support independent study.

1.3 How these notes follow the materials

The conceptual sections follow the order of the slides. The practical sections follow notebook Parts 0 through 5. Divider slides introduce the relevant sections. The closing sections cover responsible use, review questions, homework, troubleshooting, and the glossary.

Code examples retain the notebook’s calculations and settings. A few display strings use `NGN`, `C`,

and F to keep the printed source easy to compile; the notebook uses currency and degree symbols. This has no effect on the calculations. Keep the notebook's original settings for your first run, then change one setting at a time.

2 What machine learning means

Material covered: Slides 4–9.

2.1 Artificial intelligence, machine learning, and deep learning

Artificial intelligence (AI) is the broad field concerned with computer systems performing tasks associated with intelligence. Examples include recognising speech, planning a route, interpreting an image, and selecting an action in a game. AI includes several approaches, including systems built from explicitly written rules and systems trained using data.

A rule-based system might contain expert knowledge such as: if a particular combination of faults appears, recommend checking a particular component. Its behaviour follows rules specified by its designers. Learning from data is one approach within the wider AI field.

Machine learning develops behaviour through examples. For an email classifier, the training examples might be messages with labels saying whether they are spam. The learning algorithm uses those examples to build a model that can classify further messages. People still choose the data, the representation of each message, the algorithm, and the way performance is measured.

Deep learning is a part of ML that uses neural networks with multiple processing layers. A layer takes numerical inputs, combines them using adjustable values called weights, and produces another numerical representation. During training, these weights are adjusted. Multiple layers can build increasingly useful representations of complicated inputs such as images, speech, and text. The word “deep” refers to the layered structure.

Generative AI describes systems that produce content, such as text, images, audio, or code. Many familiar generative systems use deep learning. Generative modelling also has a broader history that includes other methods. The nested picture in the slide is therefore a helpful introduction to today's common systems, with some simplification in the placement of generative AI.

A single application can fit several descriptions. A neural text generator uses deep learning, which places it within ML and AI; its ability to produce text also makes it generative. A decision tree trained to classify flowers belongs to ML and AI. It uses a branching structure of questions.

2.2 What changes when we learn from examples?

Consider a simple temperature warning in ordinary programming:

```
if temperature > 80:
    print("High temperature")
```

A person selected the threshold of 80. The program evaluates that rule whenever a temperature arrives. This is useful when a clear rule is already available.

Now consider recognising handwritten digits. Different people draw a 7 with different slants, widths, and strokes. A learning approach begins with many images whose digits are known. Each image is converted into numbers, often pixel intensities. The training algorithm uses the images and labels

to find patterns associated with each digit. After training, the resulting model receives a new image and produces a digit prediction.

The supervised-learning diagram in the slides can be read in two stages:

1. **Training:** examples with known answers are supplied to a learning algorithm, which produces a trained model.
2. **Prediction:** the trained model receives a new example and estimates its answer.

The label is available during supervised training. It may be unknown when the model is used. A spam filter would have little purpose if every incoming email already arrived with a trustworthy spam label.

The phrase “the computer learns the rules” describes the outcome of a numerical procedure. Some models produce readable rules, such as a decision tree. Others represent what they have learned using arrays of numbers. Learning does not require awareness or human understanding.

2.3 A short history and why it matters

The slide places several milestones on one timeline. In 1950, Alan Turing examined the question of machine intelligence. Rosenblatt’s perceptron work in the late 1950s explored an early learning model, and Arthur Samuel used the term machine learning in 1959 in connection with his work on checkers. These examples show that the central ideas have developed over many decades.

Later milestones in the supplied timeline include Deep Blue’s 1997 chess victory, AlexNet’s influential 2012 image-classification result, AlphaGo’s 2016 match against Lee Sedol, and the public arrival of ChatGPT in 2022. The timeline includes achievements from AI more broadly. Deep Blue, for example, relied heavily on search and chess evaluation; it helps illustrate the range of methods within AI.

The dates provide context. The central lesson is that progress has come from a combination of ideas, datasets, hardware, and engineering. Modern systems build on a long history of experimentation.

2.4 Why data, computing power, and tools matter

Data provides examples. Sensors record measurements, cameras capture images, and digital services record interactions. A model needs examples relevant to the task. A large collection of badly labelled or unrepresentative examples can still produce poor results.

Computing power makes training practical. Training often involves repeatedly performing numerical calculations. A central processing unit, or CPU, handles general computation. A graphics processing unit, or GPU, can execute many suitable numerical operations in parallel. This is useful for large neural networks. The small models in the Day 1 notebook run comfortably on a CPU.

Software tools make the procedures accessible. Libraries provide reusable implementations for data handling, plots, and learning algorithms. You can concentrate on the problem while inspecting how each operation affects your data. Using a library still requires decisions about what to predict, how to test it, and whether its output is useful.

2.5 The vocabulary, using a house table

House	Size (m ²)	Rooms	Price (NGN million)	Area
1	120	3	85	Lekki
2	80	2	45	Yaba
3	200	4	250	Ikoyi
4	95	2	?	Surulere

These are invented teaching values. Each row describes one house. A **sample** is one example, so House 1 is one sample. A **dataset** is the collection of examples being used.

A **feature** is an input description available to the model. Here, size, number of rooms, and area could be features. Features can begin as numbers or categories. Many learning algorithms require numerical inputs, so a location such as Yaba usually needs an appropriate numerical encoding. A row number used only for bookkeeping is generally not a useful predictive feature.

The **target**, also called the **label**, is the answer we want to estimate. In this example it is the price. The same table could support a different task with a different target. You choose the target from the question you want to answer.

A **model** is the learned mapping from input information to an output. An **algorithm** is a procedure used to construct or operate that model. For example, a tree-learning algorithm chooses useful splits and produces a particular trained tree.

Training uses examples to establish the model. **Inference** means using the trained model to produce predictions. **Testing** compares predictions with known answers on examples reserved for evaluation. A system can perform inference without immediately knowing the true answer.

2.6 Generalisation and overfitting

We want a trained model to perform well on further examples from the situation it will encounter. This ability is called **generalisation**. A good result on training examples alone gives limited evidence of it, because those examples already influenced the model.

Overfitting happens when the model captures training-specific details, including noise, so closely that performance on new examples suffers. Imagine drawing a highly wavy curve through every measured house price. It may reproduce every training value while giving implausible prices between them. Overfitting can happen without literal memorisation of rows.

A **test set** provides an independent check when it is kept out of model development. The model predicts answers for those inputs, and we compare its predictions with the held-out labels. The split must also match the task: repeated measurements from the same person or machine may need to stay together to avoid an overly easy test.

Pause and try. For a system predicting a motor's temperature from current, voltage, and rotational speed, identify the features and target.

Answer: Current, voltage, and speed are the features. Temperature is the target. One simultaneous set of measurements could form one sample.

3 Five learning settings

Material covered: Slides 10–15.

3.1 Supervised learning

In supervised learning, training examples contain inputs and known targets. An algorithm uses the relationship between them to learn a prediction rule. The word “supervised” refers to the guidance supplied by the targets. A human does not have to watch every training step.

Suppose a table contains previous houses, their measurements, and their sale prices. Those known prices provide the training targets. If the goal is to predict price, the task is **regression**: the output is a numerical quantity whose magnitude has meaning. Other examples include temperature, energy demand, crop yield, and remaining battery life.

If the goal is to predict a category, the task is **classification**. An email might be assigned to spam or legitimate mail. A flower might be assigned to *setosa*, *versicolor*, or *virginica*. A task with two categories is called binary classification; a task with more than two categories is often called multiclass classification.

A numerical label can still represent a category. In the Iris dataset, 0, 1, and 2 are codes for species. They do not mean that species 2 is twice species 1. Likewise, recognising a handwritten digit is classification because the output selects a digit category.

The same context can give rise to different tasks. Predicting a student’s examination mark is regression. Predicting whether the student passes is classification. The question determines the target and the kind of model output you need.

The slides introduce several algorithms you will encounter later. Linear regression learns a numerical relationship for a continuous target. Logistic regression is commonly used for classification despite its name. KNN uses nearby examples. A decision tree asks a sequence of questions. A random forest combines multiple trees. A support vector machine, or SVM, learns a separating boundary. Detailed comparisons belong to later days.

3.2 Unsupervised learning

In unsupervised learning, the examples supplied for learning do not include a target answer for the intended task. The procedure looks for structure according to a mathematical objective. You still choose the inputs and method, and you still need to interpret the result.

Clustering

Clustering forms groups of similar examples. A shop might describe customers using purchase frequency and average spending, then find groups with similar behaviour. After inspecting those groups, someone might describe one as frequent low-value shoppers. That interpretation is made after inspecting the result.

Similarity depends on how examples are represented. Grouping songs using tempo and loudness can produce different groups from using lyrics and language. The choice of features therefore influences what a cluster means. K-Means in notebook Part 5 is a clustering method.

Dimensionality reduction

A feature is also called a dimension when discussing a numerical representation. A dataset with 64 feature columns has 64 dimensions. Dimensionality reduction constructs a smaller representation, perhaps two numbers per sample so that samples can be plotted on a page.

Principal component analysis, or PCA, is one method introduced later in the course. It builds new directions that capture substantial variation in the data. Reducing dimensions can make patterns easier to inspect, but some information may be lost. Two new coordinates can each combine information from several original columns.

Anomaly detection

An anomaly is an example that is unusual under the patterns a system has learned. A vibration recording that differs strongly from routine motor behaviour could trigger inspection. The unusual recording might indicate a fault, a changed operating load, or a measurement problem. Further investigation determines its meaning.

Anomaly detection can use several learning settings. Some approaches learn from unlabelled data; some learn mainly from known normal examples; others use labelled normal and abnormal examples. Its position on the slide introduces one common use of unsupervised methods.

3.3 Semi-supervised learning

Sometimes there are many examples but only a small labelled subset. For instance, a collection might contain 5,000 leaf images, of which 50 have been labelled by an expert. Semi-supervised learning uses information from both the labelled and unlabelled examples.

One approach is **pseudo-labelling**. First, train using the labelled subset. Next, use the model to produce tentative labels for unlabelled examples. Selected high-confidence predictions can be included in further training. These tentative labels can be wrong, so selection and evaluation require care. Other semi-supervised methods encourage consistent predictions when an input is slightly changed.

The practical motivation is the cost of obtaining trustworthy labels. A specialist may need time to inspect each image or recording. Unlabelled examples can be plentiful even when expert time is limited. A small independent labelled evaluation set is still valuable for measuring whether the method helps.

3.4 Self-supervised learning

Self-supervised learning constructs a training target from the data itself. Consider the sentence “The students walked into the lecture hall.” A training procedure could hide “walked” and ask a model to predict the missing word. The original sentence supplies the answer, so a person does not need to label this example separately.

Another task is predicting the next piece of text from the preceding text. Many language models use this kind of next-token objective. A **token** is a unit of text processed by a model; it can be a word, part of a word, or another text element. The missing-word example on the slide illustrates self-supervision, while the exact prediction task depends on the model.

Self-supervised tasks also apply to images and audio. A system can learn to reconstruct a hidden image region or use one part of a recording to predict another. This training can produce useful internal representations for later tasks.

For the sentence in the slide, “walked”, “ran”, and “rushed” all sound plausible. The original training sentence contains a particular observed word. A language model learns statistical patterns across many such contexts. Plausible language alone does not establish that a generated statement is factually correct.

3.5 Reinforcement learning

Reinforcement learning concerns decisions whose consequences unfold through interaction. A learner chooses actions, observes what follows, and uses a reward signal to improve its strategy. A robot learning to move through a simulated room is a useful example.

Agent

The decision-making learner, such as the robot controller.

Environment

The system it interacts with, such as the simulated room and its physics.

State or observation

Information about the current situation. The robot may observe its location, distances to obstacles, and speed. Observations may provide only partial information about the full state.

Action

A choice the agent can make, such as turning or changing wheel speed.

Reward

A numerical signal indicating the desirability of an outcome under the chosen objective.

Policy

The strategy that maps the available situation information to an action or a distribution over actions.

The agent observes its situation, chooses an action, receives a reward and new observation, and continues. Learning aims to improve the reward accumulated over time. A movement that earns little immediately may be useful if it helps reach a destination later.

Rewards can be delayed. In a game, winning at the end may depend on many earlier decisions. **Exploration** means trying actions to learn about their consequences. **Exploitation** means using actions currently believed to work well. Learning requires managing both.

Simulation can make experimentation easier and safer, although transferring a policy to physical equipment introduces differences in friction, sensing, timing, and other conditions. These differences form part of the simulation-to-reality gap. Reinforcement learning can also use previously collected interaction data; continual live interaction is not a universal requirement.

3.6 A comparison you can return to

Setting	Where the guidance comes from	Example
Supervised	Known targets paired with inputs	Flower measurements paired with species.
Unsupervised	Structure in the input data	Group customers using purchasing behaviour.
Semi-supervised	A labelled subset and additional unlabelled examples	Expert-labelled leaf images alongside many unlabelled images.
Self-supervised	A target constructed from existing data	Predict the next token from preceding text.
Reinforcement	Rewards associated with actions and outcomes	Learn a navigation policy through experience.

These settings can appear together in one system. A model may first learn representations through self-supervision, then train on labelled examples for a particular task. They describe the source of learning guidance rather than five completely separate products.

Pause and try. Identify the learning setting: grouping unlabelled messages by topic; predicting transformer failure from labelled past records; learning drone control from rewards; using 50 labelled images alongside 4,950 unlabelled images.

Answers: Unsupervised clustering; supervised classification; reinforcement learning; semi-supervised learning.

4 The ML workflow, from question to usable system

Material covered: Slide 16.

4.1 Define the problem

Start with a question precise enough to test. “Use AI on a motor” leaves many decisions open. “Use the last five seconds of vibration to predict whether this motor has a bearing fault” identifies an input, a prediction target, and a time interval.

Also ask who will use the result and what action follows. If the output triggers an inspection, an unnecessary alert costs time. Missing a developing fault has a different cost. Those consequences affect how performance should be judged.

4.2 Collect suitable data

Collect examples representing the situations the system will face. For the motor, record relevant operating speeds, loads, sensors, and conditions. For supervised learning, establish how the target labels were obtained. A label based on a technician’s confirmed inspection has a different meaning from an unverified guess.

Keep enough information to trace examples to their sources. If hundreds of windows come from one recording, those windows have a shared origin. This matters when splitting the data. Data quantity, measurement quality, coverage, and label reliability all affect the experiment.

4.3 Explore the data and plan the split

Exploratory data analysis (EDA) means examining the data to understand its content and identify issues. Look at column names, units, missing entries, typical ranges, category counts, and plots. In Iris, counting species reveals whether one species is much more common than the others.

Plan the evaluation split early. A random split is appropriate for this small introductory Iris exercise. Future-value prediction often needs a time-based split. A system intended for new people may need a person-based split. The split should reflect what “new” means in the intended application.

4.4 Clean the data and construct features

Cleaning may include correcting known unit errors, handling missing values, checking repeated rows, or resolving inconsistent labels. An unusual value deserves investigation. It could be a valid rare event.

Feature construction converts raw information into useful inputs. For a vibration recording, you might calculate mean amplitude, variability, or energy in particular frequency ranges. A different model might work directly with a sequence of signal samples. These choices affect what information is available to the model.

Data leakage occurs when information unavailable during genuine prediction enters model development. Using the target as an input is one obvious example. Learning preprocessing values from held-out data can also leak information. Fit learned preprocessing on the training portion, then apply that fitted transformation to the held-out portion. Scikit-learn pipelines help organise this separation.¹

4.5 Train and evaluate

Choose an algorithm and fit it using training data. A simple initial model gives you a **baseline**: a result against which further work can be compared. For a classifier, even always choosing the most common class provides a basic reference score.

Use a validation set, or validation folds, to compare settings during development. A **hyperparameter** is a setting chosen outside the fitting process, such as the number of neighbours in KNN. Keep the final test set separate from this decision-making. Repeatedly choosing settings because they score best on the test set weakens its independence.

Evaluation should consider the intended question and the kinds of mistakes. Accuracy counts the proportion of correct classifications. Numerical prediction needs an error measure suited to numbers. Later days introduce further metrics and more reliable comparisons.

4.6 Deploy, monitor, and improve

Deployment makes a trained model available in a working application. This can mean a web service, a desktop dashboard, a mobile application, or firmware on a device. The application must collect the expected inputs and apply compatible preprocessing before asking the model for a prediction.

After deployment, input conditions can change. A sensor can age, a machine can operate at a different load, or users can behave differently. Such changes may reduce performance. Monitoring includes checking input quality, prediction behaviour, and, when true outcomes become available, actual errors. Retraining is useful when supported by new data and evaluation.

¹See the official common-pitfalls reference in the final section.

The workflow is iterative. A weak result may send you back to inspect the labels, change the features, or collect missing examples. Training is one part of a wider engineering process.

5 Where these ideas appear in practice

Material covered: Slides 17–20.

5.1 Computer vision

Computer vision works with images and video. A digital image is a grid of pixel values. A colour image usually stores several colour-channel values at each location. These values can become inputs to a model.

Image classification gives an image a category, such as a leaf condition. **Object detection** identifies objects and their locations, often using rectangles called bounding boxes. **Segmentation** assigns categories to individual pixels or regions. These tasks require different forms of labelled data and different outputs.

The slides mention face recognition, number plates, crop images, and vehicles. Each application also depends on image quality, lighting, viewpoint, and the examples used during training. A leaf model trained on clean photographs may need further testing on cluttered field images.

5.2 Bioinstrumentation and physiological signals

Bioinstrumentation involves measuring signals from the body. An electrocardiogram (ECG) records electrical activity associated with the heart. An electroencephalogram (EEG) measures electrical activity associated with the brain, usually through electrodes on the scalp. An electromyogram (EMG) measures electrical activity associated with muscle activation.

A signal is a sequence of measurements over time. A **window** is a selected time segment, such as five seconds of recording. Features can describe that segment's amplitude, timing, or frequency content. A model may use those features to estimate a category or numerical output.

Engineering examples include studying rhythm patterns in ECG, recognising muscle-activation patterns for a prosthetic controller, or relating EEG patterns to a defined experimental task. Such outputs require evaluation under the conditions in which they will be used. Sensor contact, motion, noise, and differences between people affect the data.

5.3 Robotics, finance, agriculture, language, and transport

A robot may use a vision model to locate an object, a planning method to choose a path, and a controller to move its motors. Some components can use ML, while others use established equations and rules. Learning-based components still need to work with the robot's physical constraints.

In finance, models can estimate transaction risk or group spending patterns. In agriculture, they can estimate crop yield or analyse plant images. Recommendation systems use information about users and items to estimate which items might be relevant. A recommendation is a prediction under a chosen objective, such as the likelihood of watching or purchasing.

Language applications include speech recognition, translation, and text generation. Transport applications can estimate travel time or demand using past observations and current conditions. In each case, ask the same three questions: what is the input, what output is desired, and what examples could support learning that relationship?

5.4 TinyML and edge AI

Edge AI means running AI computations near the source of the data, for example on a phone, camera, or local device. **TinyML** commonly concerns small ML models on highly constrained devices such as microcontrollers. An ESP32 running a small sound classifier is a useful example. A Raspberry Pi is a small general-purpose computer with a different resource budget.

Local inference can avoid a network round trip, support operation without continuous connectivity, and reduce the need to transmit raw data. The actual latency and power consumption depend on the hardware, model, sensing, and communication workload. There is no universal 200-millisecond cloud delay. Keeping inference local can help privacy, while storage and transmission choices still matter.

An embedded workflow often contains these stages:

1. Train and evaluate a model on a computer using representative data.
2. Convert it into a format supported by the target device's inference software.
3. Where appropriate, quantise the model and check how this changes size, speed, and accuracy.
4. Integrate preprocessing and inference into firmware, load it onto the device, and test the complete system.

Quantisation represents numerical values using lower precision. Moving from 32-bit floating-point weights to 8-bit values can reduce the storage required for those weights substantially. The complete application's memory reduction depends on other stored information and runtime needs. Accuracy can change and should be measured again.

Flashing a board means writing the program to its non-volatile memory. The program usually contains more than a model: it also reads sensors, prepares inputs, invokes inference, and handles outputs. Supported operations, working memory, battery use, and timing must all fit the device.

Pause and try. Choose an application from your department. Write down its inputs, desired output, likely learning setting, and one measurement difficulty.

Example: For motor-condition classification, the input could be a vibration window, the output could be healthy or faulty, the setting could be supervised learning, and changing motor load could complicate the measurements.

6 Python, the toolbox, and your notebook

Material covered: Slides 21–25; notebook Part 0.

6.1 Why Python is used throughout the course

Python is a programming language with readable syntax and a large collection of scientific tools. A **library** is a collection of reusable code. Importing a library makes its functions and objects available to your program. Many scientific libraries perform their demanding calculations through compiled numerical routines.

For example, the short program in the slide stores four marks in a list, adds them, counts them, and divides the total by the count:

```
scores = [65, 72, 88, 91]
average = sum(scores) / len(scores)
print("Average:", average)
```

The total is 316 and there are four marks, so the average is $316/4 = 79$. The assignment sign = associates the result with the name **average**. The function **print** displays the result. Python's division operator / returns a floating-point result here, so the output is **Average: 79.0**.

6.2 The tools and what each one contributes

Tool	Role in the course
NumPy	Numerical arrays and mathematical operations on collections of values. Used in the house-price learning loop.
Pandas	Labelled tables, column selection, summaries, and data preparation. Its main table object is called a DataFrame.
Matplotlib	Building figures, axes, lines, labels, and other plotting elements.
Seaborn	Statistical plots that work conveniently with labelled tables, including the Iris scatter plot.
scikit-learn	Implementations of many conventional ML algorithms, dataset loaders, train/test splitting, and evaluation tools. Its Python import name is sklearn .
TensorFlow and PyTorch	Frameworks for constructing and training neural networks.
OpenCV	Tools for working with images and video, including many operations beyond ML.
YOLO	A family of object-detection approaches; available implementations provide trained models and training tools.
Gymnasium	A common interface to environments used in reinforcement-learning experiments.
Hugging Face	An ecosystem of model-sharing resources and libraries, including tools for language and other model families.
TensorFlow Lite tooling	The lightweight deployment tooling named in the slides, used for supported mobile and embedded inference workflows. Exact runtime compatibility depends on the target.

Tool	Role in the course
Edge Impulse	A platform for building and deploying supported embedded ML workflows.

Some entries are Python libraries, others are model families or platforms. You will use them when their role becomes relevant. For today, recognise the path from a table in Pandas to a plot in Seaborn and a trained model in scikit-learn.

6.3 What Google Colab is doing

Google Colab provides a notebook interface in your browser. A notebook combines explanations, executable code, and outputs. Its file extension is `.ipynb`. The code runs in a runtime, which is the computing session containing Python and the values created by your cells.

A saved notebook preserves its code and can preserve visible outputs. The runtime's live variables are separate. If the runtime resets, you need to rerun the cells that create those variables. This is why a visible old output can remain in a notebook even when its associated variable no longer exists in the current session.

Open the supplied notebook in Colab and save an editable copy to your Drive if it is shared as a view-only document. Run code cells with the play button or **Shift + Enter**. Work from top to bottom on your first run. Colab's free resources have usage limits, and GPU access is not guaranteed. Day 1 does not require a GPU.²

6.4 Your first two cells

```
print("Hello, Machine Learning!")
```

The quoted text is a **string**. Parentheses contain the information passed to `print`. When the cell runs, the message appears underneath it. The supplied notebook adds a waving-hand emoji to the greeting.

The next cell contains:

```
2 + 3
```

Its result is 5. A notebook can display the value of the last expression in a cell automatically. Assignment statements usually do not display a result by themselves. In a regular Python script, you would generally use `print(2 + 3)` to display this value.

²Google Colab FAQ, listed in the references.

6.5 A small guide to the punctuation

Symbol	Meaning in the examples
=	Assign a value to a name, or supply a named argument inside a call.
==	Compare whether two values are equal.
()	Call a function or method, or group part of an expression.
[]	Create a list, select an item, or select a slice, depending on context.
{}	Curly braces create dictionaries; in an f-string, braces surround an expression to insert.
:	Begins an indented block after <code>if</code> , <code>for</code> , or <code>def</code> ; also appears in slices and dictionaries.
.	Access an attribute or method, as in <code>df.shape</code> or <code>temps.append(30)</code> .
##	Starts a comment extending to the end of the line.
*, /, **	Multiplication, division, and raising to a power.

A **function** is a callable operation such as `len(temps)`. A **method** is an operation attached to an object, such as `temps.append(30)`. An **attribute** is information accessed through an object, such as `df.shape`. Parentheses usually tell you that an operation is being called.

7 Python essentials, explained line by line

Material covered: Slide 23; notebook Part 1, including Exercise 1.

7.1 Variables and types

```
name = "Ada"
age = 20
gpa = 4.6
is_student = True

print(name, age, gpa, is_student)
print(type(name), type(age), type(gpa), type(is_student))
```

A variable name lets you refer to a value later. `name` refers to the string `"Ada"`; `age` refers to the integer 20; `gpa` refers to the floating-point number 4.6; and `is_student` refers to a Boolean value.

A **string**, written `str`, stores text. An **integer**, written `int`, represents a whole number. A **float** represents a number using floating-point arithmetic, which commonly includes fractional values. A **Boolean**, written `bool`, represents `True` or `False`. Capitalisation matters: `True` is the Boolean spelling used by Python.

The first print statement displays `Ada 20 4.6 True`. The second asks Python for each value's type. Quotes distinguish text from variable names: `print("name")` displays the word `name`; `print(name)` displays the value associated with the variable.

7.2 Putting values into a sentence

```
print(f"{name} is {age} years old and has a GPA of {gpa}")
```

The `f` before the quote creates an f-string. Python evaluates each expression inside braces and inserts its value into the text. The output is `Ada is 20 years old and has a GPA of 4.6`. If you change `age` and rerun this cell, the inserted age changes accordingly.

Later, a format such as `{accuracy:.2f}` displays a number with two digits after the decimal point. Formatting changes its printed appearance; it does not change the stored value used in calculations.

7.3 Lists, indexing, and slicing

```
temps = [31, 29, 33, 35, 28]

print("First day:", temps[0])
print("Last day:", temps[-1])
print("First three days:", temps[0:3])
print("Number of days:", len(temps))

temps.append(30)
print("After append:", temps)
```

A list stores values in order. The square brackets in the first line create the list. The values describe five temperatures in degrees Celsius. Each item has a position called an **index**. The first index is zero:

Index	0	1	2	3	4
Value	31	29	33	35	28

Thus `temps[0]` gives 31. A negative index counts backwards from the end, so `temps[-1]` gives 28 at that point in the cell. The slice `temps[0:3]` starts at index 0 and stops before index 3. Its result is `[31, 29, 33]`. The end index is excluded.

The function `len(temps)` returns the number of items, which is initially 5. The method `append(30)` changes the existing list by adding 30 at the end. After this operation, the list is `[31, 29, 33, 35, 28, 30]`, and its length is 6.

Execution order explains why the displayed last day is initially 28 even though the final list ends in 30: the first print happened before the append. If you rerun the whole cell, the first line resets the list before appending. If you run an extra standalone append cell repeatedly, the list keeps gaining entries.

7.4 Dictionaries and named values

```
student = {"name": "Ada", "level": 200,
           "department": "Electrical Engineering"}

print(student["name"])
print(student["level"])

student["gpa"] = 4.6
print(student)
```

A dictionary stores **key–value pairs**. In `"level": 200`, the key is `"level"` and the value is 200. The keys identify the information being stored. `student["name"]` retrieves Ada, and `student["level"]` retrieves 200.

Assigning `student["gpa"] = 4.6` adds a new entry. If that key already existed, the assignment would replace its value. The same idea appears when selecting a named column from a Pandas table. A dictionary can represent more general mappings as well as one record.

A missing dictionary key raises a `KeyError`. For example, `student["height"]` fails here because no height entry was created. Check exact spelling and capitalisation when retrieving a key.

7.5 Loops and conditions

```
for t in temps:
    if t > 30:
        print(t, "C -> hot")
    else:
        print(t, "C -> okay")
```

The loop takes each value from `temps` in order and temporarily calls it `t`. On the first pass, `t` is 31. Python evaluates `t > 30`, which is true, and prints the hot message. On the second pass, `t` is 29, so the `else` branch runs.

The comparison is strictly greater than 30. The final value of 30 therefore produces the okay message. Using `>=` would include equality and change that result.

Indentation shows which statements belong inside each block. The `if` line is inside the loop, and its print statement is inside the conditional branch. Four spaces per level is the standard convention used here; Python requires consistent indentation. A colon introduces each block. Misaligned lines can cause an error or change which statements repeat.

This cell uses a manually selected temperature threshold. It demonstrates programming control flow. Later, the decision-tree algorithm selects thresholds from training examples.

7.6 Functions and return values

```
def to_fahrenheit(c):
    return c * 9 / 5 + 32

print(to_fahrenheit(30))
print(to_fahrenheit(100))
```

The keyword `def` defines a function. The name `to_fahrenheit` describes its purpose. The parameter `c` is a local name for the Celsius value supplied when the function is called. The expression follows

$$F = \frac{9}{5}C + 32.$$

For $C = 30$, the calculation is $30 \times 9/5 + 32 = 86$. For $C = 100$, it is 212. The displayed results are 86.0 and 212.0.

The keyword `return` gives the calculated value back to the caller. The surrounding `print` displays that returned value. Returning a result also allows another calculation to use it, as in `reading_f = to_fahrenheit(30)`. Defining a function makes it available; its body runs when you call it.

7.7 Exercise 1: full solution and reasoning

The first task is to convert every temperature. Start with a new empty list and add one converted value for each original value:

```
temps_f = []
for t in temps:
    temps_f.append(to_fahrenheit(t))
print("In F:", temps_f)
```

On each pass, `to_fahrenheit(t)` returns one converted value. `append` adds that value to `temps_f`. The Celsius list remains available. After the six passes, the converted list is:

```
[87.8, 84.2, 91.4, 95.0, 82.4, 86.0]
```

The next task defines a function for the arithmetic mean:

```
def average(numbers):
    return sum(numbers) / len(numbers)

print(f"Average temperature: {average(temps):.1f} C")
```

The parameter `numbers` can refer to any suitable non-empty list of numerical values. In this example, the six Celsius readings sum to 186, so their mean is $186/6 = 31.0$. The format `.1f` prints one decimal place.

The function assumes a non-empty list. An empty list would give a length of zero and cause division by zero. More complete programs can check for that situation. Here, the exercise is designed around the six readings already created.

Pause and try. What would the average be if you used the original five readings before appending 30?

Answer: Their sum is 156, so the average is $156/5 = 31.2$. A different notebook state changes the result even when the averaging function is correct.

8 Meeting the Iris dataset

Material covered: Notebook Part 2; vocabulary from slide 9.

8.1 What the measurements describe

The dataset used in the notebook contains 150 iris flowers from three species: *setosa*, *versicolor*, and *virginica*. Each flower has four measurements in centimetres: sepal length, sepal width, petal length, and petal width. A sepal is an outer floral part that protects a developing bud; a petal is another floral part, often associated with the flower's visible display.

For our task, one flower is one sample. Its four measurements are the features. Its species is the target. The dataset includes 50 flowers of each species, so its class counts are balanced. Balanced counts make one aspect of evaluation simpler, although they do not guarantee that all classes are equally easy to recognise.

8.2 Loading the table

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df["species"] = df["target"].map(
    dict(enumerate(iris.target_names))
)

df.head()
```

`import pandas as pd` imports Pandas and assigns it the conventional short name `pd`. The `from ... import ...` statement imports a particular function, `load_iris`, from scikit-learn's dataset tools.

Calling `load_iris(as_frame=True)` loads the built-in dataset and requests Pandas structures. The returned object contains data as well as descriptive information. Its `frame` attribute gives a table containing the four feature columns and a numeric target column. We associate that table with `df`.

The longer assignment creates a readable species column. Read it from the inside out:

1. `iris.target_names` contains the three species names in label order.
2. `enumerate(...)` pairs each name with its position: 0, 1, or 2.
3. `dict(...)` turns those pairs into a mapping such as 0: "setosa".
4. `df["target"].map(...)` replaces each target code with its corresponding species name.
5. `df["species"] = ...` stores those names in a new column.

The label mapping is:

Code	Species
0	setosa
1	versicolor
2	virginica

`df.head()` displays the first five rows by default. In a notebook, the final expression displays as a formatted table. `head` is a preview; it does not delete the remaining rows.

8.3 Checking dimensions and counts

```
print("Shape (rows, columns):", df.shape)
print()
print("Features:", iris.feature_names)
print("Label names:", [str(n) for n in iris.target_names])
print()
print("How many flowers of each species?")
print(df["species"].value_counts())
```

The `shape` attribute reports (150, 6). There are 150 rows and six columns: four measurements, the numeric target, and the species text added above. Only the four measurements are intended as features. Including the target or species among the classifier's inputs would reveal the answer.

`iris.feature_names` provides the measurement names. The list expression in the following print statement is a **list comprehension**: it goes through each name, converts it to a Python string, and builds a list. This is a compact way to construct a list using a loop-like expression.

`value_counts()` counts how many times each species appears. It returns 50 for each one. An empty `print()` produces a blank line for readability. These statements inspect the dataset; no model has been trained yet.

8.4 Reading the scatter plot

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(data=df, x="petal length (cm)",
                y="petal width (cm)", hue="species", s=60)
plt.title("Iris flowers: petal size by species")
plt.show()
```

The aliases `plt` and `sns` refer to plotting tools. `data=df` supplies the table. The `x` and `y` arguments select the two columns plotted along the horizontal and vertical axes. `hue="species"` uses a separate colour for each species. `s=60` sets marker size, and `plt.title` supplies the title.

Each dot represents one flower's two selected measurements. Nearby dots have similar petal measurements. The legend tells you which species each colour represents. Several flowers can have the same or nearly the same coordinates, so dots can overlap.

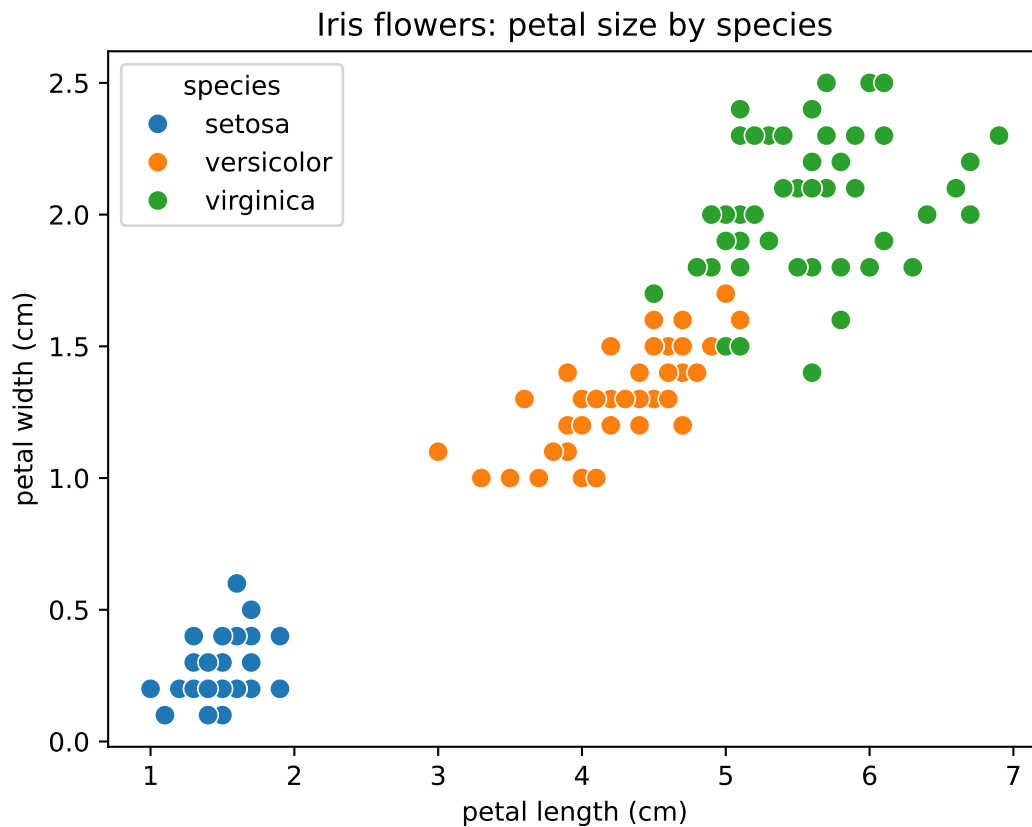


Figure 1: The scatter plot produced by notebook Part 2. Each point represents one flower.

Setosa forms a visibly separate group with relatively short, narrow petals. Versicolor and virginica have some overlap. A flower with a petal length of 1.5 cm would therefore suggest setosa in the context of this dataset. That observation is an initial pattern found by inspecting examples.

The plot shows two of the four available features. A later model may use all four. The class colours are supplied by the known labels for inspection; the plot itself is not a trained classifier.

9 Watching a model learn, one calculation at a time

Material covered: Slide 8; notebook Part 3.

9.1 The six invented houses

The notebook uses these teaching values:

House	Size (m ²)	Price (NGN million)
1	50	26
2	80	41
3	100	49
4	120	61
5	150	74
6	200	101

We choose the model

$$\hat{y} = wx,$$

where x is size, $\hat{y}$ is predicted price, and w is an adjustable coefficient. The hat over y indicates a prediction. The actual observed price is written y .

The model has one adjustable **parameter**, w . Its units here are NGN million per square metre. If $w = 0.5$, a 100-square-metre house receives a predicted price of 50 million naira.

This simple model has no constant offset, also called an intercept. Its line passes through zero. A more general straight-line model would use $\hat{y} = wx + b$, where b is an intercept. Real property prices also depend on many factors absent from this example. Here the narrow model lets us see the learning procedure clearly.

9.2 Arrays and the starting settings

```
import numpy as np

size = np.array([50, 80, 100, 120, 150, 200])
price = np.array([26, 41, 49, 61, 74, 101])

w = 0.1
learning_rate = 0.00002
errors = []
```

A NumPy array stores numerical values in a structure designed for numerical operations. The first item in `size` matches the first item in `price`, and this pairing continues through the arrays. Keeping inputs and targets aligned is essential.

`w = 0.1` gives the initial coefficient. The **learning rate** controls the size of each adjustment. The list `errors` will record the error at each step so that we can plot progress.

Multiplying a NumPy array by a number multiplies every element. Thus `w * size` produces six predictions at once. A Python list behaves differently: multiplying a list by an integer repeats its contents. NumPy's element-by-element arithmetic is useful throughout ML.

9.3 Making the first predictions

With $w = 0.1$, the predicted prices are:

$$[5, 8, 10, 12, 15, 20].$$

These predictions are all below the corresponding observed prices. For the first house, the difference is $5 - 26 = -21$. The sign tells us that the prediction is lower than the target. This signed difference is often called a residual, with the chosen sign convention stated explicitly.

We need one measure summarising all six errors. The notebook uses **mean squared error**, abbreviated MSE:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (wx_i - y_i)^2.$$

Here n is the number of examples, which is six. The index i identifies an example. The summation symbol tells us to add a quantity across the examples. For each example, subtract the actual price from the predicted price, square the difference, add all the squared differences, and divide by six.

Squaring makes each contribution non-negative and gives larger mistakes more influence. At the first step, the residuals are $[-21, -33, -39, -49, -59, -81]$. Their squares sum to 15,494, so

$$L(0.1) = \frac{15,494}{6} \approx 2582.33.$$

Because prices are measured in NGN millions, MSE has units of squared NGN millions. An MSE of 0.93 is not directly a price error of 0.93 million. Taking its square root gives the root mean squared error, about 0.96 million here.

9.4 How the adjustment is calculated

A **loss function** assigns a number to prediction errors. Training seeks parameter values that reduce that number. **Gradient descent** uses the slope of the loss with respect to the parameters to decide how to adjust them.

For this one-parameter model, the derivative is

$$\frac{dL}{dw} = \frac{1}{n} \sum_{i=1}^n 2(wx_i - y_i)x_i.$$

A derivative describes how a quantity changes when another quantity changes slightly. A negative derivative means that increasing w locally reduces the loss. A positive derivative means that decreasing w locally reduces it. The update is

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{dL}{dw},$$

where α is the learning rate. At the initial value, the derivative is approximately $-12,841.67$. Therefore,

$$w_{\text{new}} = 0.1 - (0.00002)(-12,841.67) \approx 0.35683.$$

The coefficient increases, which raises the next set of price predictions. Repeating the process gradually brings it close to 0.50209.

If the calculus is new, focus first on the role of each part: the residual describes the prediction error, the gradient gives an adjustment direction, and the learning rate controls the amount moved. The derivative formula can be revisited when you study optimisation more deeply.

9.5 Every line of the learning loop

```
for step in range(15):
    guess = w * size
    error = np.mean((guess - price) ** 2)
    errors.append(error)
    w = w - learning_rate * np.mean(2 * (guess - price) * size)
    print(f"step {step:2d}:  w = {w:.4f}    error = {error:10.2f}")
```

`range(15)` supplies integers 0 through 14, so the body runs 15 times. `guess = w * size` calculates all six predictions using the current coefficient. `guess - price` subtracts the targets element by element. The operator `** 2` squares each residual, and `np.mean` averages the six squared values.

`errors.append(error)` stores the current loss. The following line applies the gradient update using the current predictions. Assigning back to `w` replaces the old coefficient with the updated coefficient for the next pass.

The print statement formats the step as an integer in a field of width two, displays w to four decimal places, and displays the error to two decimal places in a field of width ten. These widths align the output columns.

A detail in the printed table: each row displays the updated w alongside the error calculated before that update. For example, row 0 displays $w = 0.3568$ with error 2582.33, but that error was calculated using the starting $w = 0.1$. The next row's error reflects the updated coefficient. This follows directly from the order of the statements.

Step	Updated w shown	Loss before that update
0	0.3568	2582.33
1	0.4496	337.96
2	0.4831	44.93
3	0.4952	6.68
4	0.4996	1.68
5	0.5012	1.03
7	0.5020	0.93
14	0.5021	0.93

9.6 Reading the plots and predicting another house

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(errors, marker="o")
axes[0].set_title("Error goes down as the model learns")
axes[0].set_xlabel("step")
axes[0].set_ylabel("error")

axes[1].scatter(size, price, label="real houses", s=60)
axes[1].plot(size, w * size, color="orange", linewidth=3,
             label=f"learned line (w = {w:.3f})")
axes[1].set_title("The learned model")
axes[1].set_xlabel("size (m2)")
axes[1].set_ylabel("price (NGN millions)")
axes[1].legend()
plt.tight_layout()
plt.show()

new_house = 90
print(f"Predicted price: NGN {w * new_house:.1f} million")
```

`plt.subplots(1, 2)` creates one row of two panels. The `fig` object represents the whole figure; `axes[0]` and `axes[1]` refer to the individual panels. `figsize=(11, 4)` requests a figure 11 inches wide and 4 inches high.

The left panel plots the saved errors against their positions in the list. The circles mark recorded values. The right panel shows the observed house data as dots and the model's predictions as a line. Titles identify each panel, axis labels give meaning to the coordinates, and the legend explains the two types of marks. `tight_layout()` adjusts spacing around plot elements.

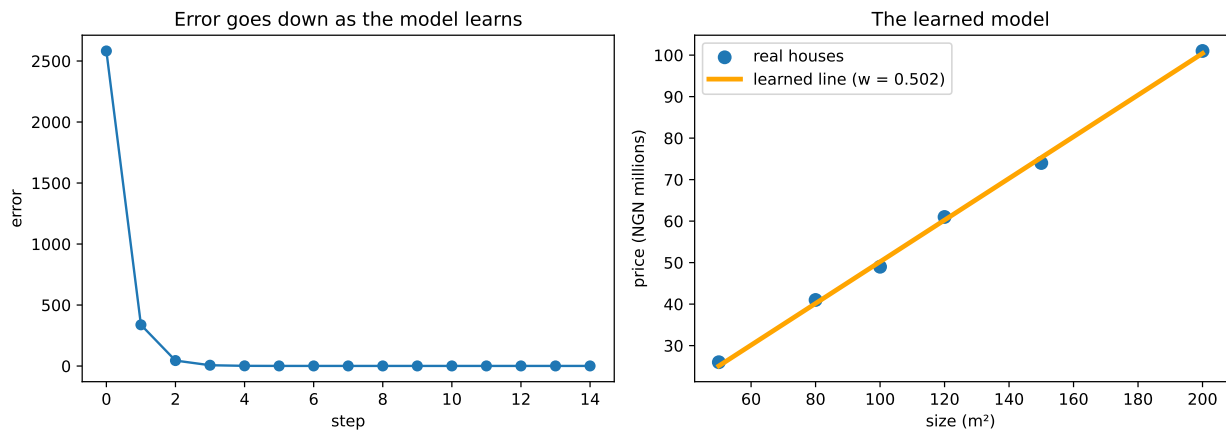


Figure 2: Training loss and the fitted house-price line, generated from the supplied notebook.

The final coefficient is approximately 0.50209. For a 90-square-metre house, the calculation is $0.50209 \times 90 \approx 45.19$, displayed as NGN 45.2 million. This is inference using the learned coefficient.

The falling curve shows improvement on the six training houses. The notebook does not reserve separate houses to test this particular model. The plot therefore demonstrates optimisation; it does not establish real-world house-price accuracy.

9.7 Changing the learning rate and starting value

Changing the learning rate to 0.0001 makes each adjustment five times larger. With the original unscaled house sizes, that is too large for this update rule. The coefficient repeatedly overshoots, the predictions become extreme, and the loss grows. In the checked run, the final printed loss exceeded nine trillion in the example's squared units.

Starting with $w = 5$ while retaining the original learning rate of 0.00002 produces a very large initial error, but the updates still bring w close to 0.50209 after 15 steps. This particular loss is a simple convex quadratic, so a suitable step size works well from either starting value. More complex models can behave differently.

Reset `w`, `learning_rate`, and `errors` before comparing runs. Change one choice at a time so that you can explain the cause of the difference. The effect of a learning rate also depends on input scale; converting square metres to a different unit changes the numerical gradients.

9.8 How widely this learning loop applies

Gradient-based optimisation is central to neural-network training and many other methods. The Day 1 decision tree learns by choosing splits, KNN retains training examples for later neighbour searches, and K-Means alternates cluster assignment with centre updates. Each has its own learning procedure. The general aim is to find useful structure under the chosen method and objective.

10 Training the first flower classifier

Material covered: Slide 26; notebook Part 4 and Parts 4.1–4.3.

10.1 Load the inputs and targets

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
```

The three imports provide a dataset loader, a splitting function, and a decision-tree classifier. Calling `load_iris(return_X_y=True)` requests the input measurements and target codes directly. The two returned values are assigned to `X` and `y`.

Uppercase `X` conventionally names a feature matrix: a table of numerical inputs. Here its shape is $(150, 4)$, meaning 150 flowers and four measurements per flower. Lowercase `y` names the target array, with one species code for each flower. Its shape is $(150,)$. These names are conventions that make code familiar to other ML users.

The i th row of `X` and the i th value of `y` describe the same flower. The numeric target is kept outside the input matrix. This preserves the intended task of predicting species from measurements.

10.2 Split the flowers into training and test sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training flowers:", len(X_train),
      "| Test flowers:", len(X_test))
```

`test_size=0.2` sets aside 20 percent of the 150 flowers, giving 30 test examples and 120 training examples. The splitting function keeps measurements and labels paired as it divides them. Its four outputs are assigned in the order shown:

Variable	Contents	Shape
<code>X_train</code>	Training measurements	$(120, 4)$
<code>X_test</code>	Test measurements	$(30, 4)$
<code>y_train</code>	Training species codes	$(120,)$
<code>y_test</code>	Test species codes	$(30,)$

`random_state=42` fixes the random seed used for this split. In the same software environment and with the same inputs, this helps reproduce the same selected rows. The number 42 has no special predictive ability; another fixed integer could serve the same purpose.

The original exercise uses an ordinary shuffled split without an explicit `stratify` argument. Stratification is an option that helps preserve class proportions across splits. That is useful later, especially when some classes are uncommon. We retain the original split here so that the notebook and notes agree.

10.3 Create and fit the model

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

The first line creates a classifier object with the chosen settings. At this point, it has not learned the flower dataset. The second line performs fitting using the training measurements and training targets.

A decision tree builds a sequence of feature-based questions. A question could be whether petal length is at most 2.45 cm. Each answer sends an example down a branch to another question or a terminal node called a **leaf**. A leaf supplies a predicted class based on the training examples that reached it.

During fitting, the algorithm compares possible splits and chooses ones that improve separation of the classes according to its criterion. The resulting thresholds and branches are learned from the training examples. The notebook also fixes the classifier's random state to help reproduce its choices. The short slide example omits this second seed; the expanded notebook includes it.

10.4 Score the held-out examples

```
accuracy = model.score(X_test, y_test)
print(f"Accuracy on unseen flowers: {accuracy:.2f} "
      f"({accuracy*100:.0f}%)")
```

For this classifier, `score` returns mean classification accuracy:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of evaluated examples}}.$$

The verified run correctly classifies all 30 test flowers, so the score is $30/30 = 1.0$, displayed as 100 percent. If 27 had been correct, the score would have been $27/30 = 0.9$, or 90 percent.

The `score` method's meaning depends on the estimator. Many classifiers use accuracy; regression estimators commonly use another measure such as R^2 . Always check the relevant definition when changing the task or estimator. The classifier documentation identifies accuracy as the score used here.

10.5 What 100 percent on this split tells us

This result describes 30 particular held-out examples under the specified split. The dataset is small, clean, and contains measurements that separate the species fairly well. A different split or a different collection of flowers can produce another result. A measured 100 percent test score does not imply guaranteed accuracy on every future flower.

A useful habit is to report the denominator with the percentage: "30 of 30 test flowers were classified correctly." It makes the amount of evidence visible. Real applications also need data representative of their intended use and checks on important subgroups and error types.

There is no universal accuracy threshold that makes an ML system good. A baseline, class balance, error costs, and the conditions of evaluation all matter. If 99 percent of examples belong to one class, always predicting that class already gives 99 percent accuracy while failing to recognise every example from the other class.

10.6 Compare predictions with their true labels

```
predictions = model.predict(X_test)

results = pd.DataFrame({
    "real species": iris.target_names[y_test],
    "predicted": iris.target_names[predictions],
})
results["correct?"] = (
    results["real species"] == results["predicted"]
)
results.head(10)
```

`predict(X_test)` produces one class code per test flower. The indexing operation on `iris.target_names` uses the numeric codes in `y_test` to retrieve species names. Applying the same operation to `predictions` converts the predicted codes into names.

`pd.DataFrame` builds a table from the dictionary. Each dictionary key becomes a column name and each associated sequence supplies the column's values. The equality comparison `==` operates row by row on the two columns. A `True` value means the names match for that flower; `False` indicates a mistake.

`results.head(10)` displays the first ten comparisons. All ten displayed comparisons are true in the original run, and the full test set contains 30 correct predictions. The table helps you connect the single accuracy value with individual examples. In a harder task, inspecting the incorrect rows can reveal recurring problems.

10.7 Predict a new flower

```
new_flower = [[5.0, 3.4, 1.5, 0.2]]
pred = model.predict(new_flower)[0]
print("This flower is probably:", iris.target_names[pred])
```

The four values must appear in the training feature order: sepal length, sepal width, petal length, and petal width. Their units must also match the training data, here centimetres.

There are two pairs of square brackets because the model expects a collection of samples. The outer list is the collection, and the inner list is one flower's four features. Its shape is one row by four columns. A batch of three flowers would contain three inner lists.

`predict` returns an array with one prediction because one flower was supplied. The following `[0]` extracts its first value. That value is used to look up the species name. The result is `setosa`, consistent with the short, narrow petal measurements.

The word “probably” is part of the print message. This cell prints a predicted class, with no numerical probability attached. The model also has no built-in guarantee that an arbitrary input actually describes one of these three iris species. Input validity still matters.

10.8 Drawing and reading the tree

```
from sklearn.tree import plot_tree

plt.figure(figsize=(14, 7))
plot_tree(model, feature_names=iris.feature_names,
          class_names=list(iris.target_names), filled=True,
          rounded=True, fontsize=9, max_depth=2)
plt.title("The first rules the decision tree learned")
plt.show()
```

`plot_tree` draws the fitted model. `feature_names` replaces feature indices with readable measurement names. `class_names` supplies readable class labels. `filled=True` colours nodes according to their class composition, and `rounded=True` rounds their boxes. The font size and figure size control appearance.

`max_depth=2` in this plotting call limits how much of the existing tree is displayed. It does not change the fitted model. The root is depth zero, so the display includes the root and two further depths, with deeper structure truncated. To restrict learning itself, a depth limit would need to be supplied when creating the classifier.

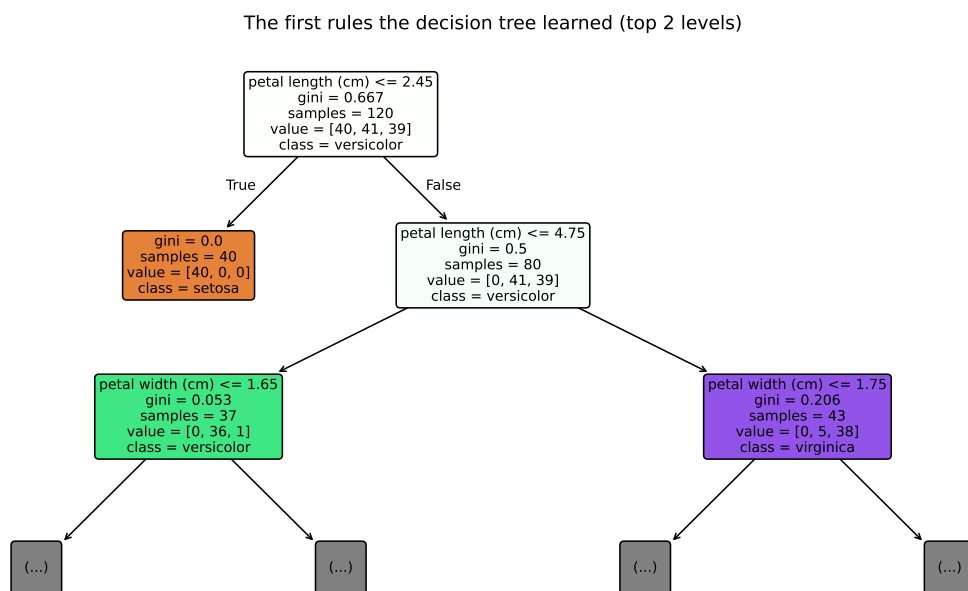


Figure 3: The upper portion of the fitted decision tree. The display depth is limited; the full fitted tree may continue further.

At the root of the checked model, the question is whether petal length is at most approximately 2.45 cm. A true answer follows the left branch. The 40 training examples on that side are setosa. Longer petals go to the other branch, where additional questions separate the remaining species.

The node labels include several quantities:

Split rule

The feature and threshold used at that node, when the node has a split.

Samples

The number of training examples reaching the node.

Value

The class composition at the node in the class order. Depending on the installed scikit-learn version, the plot can express this using counts or proportions.

Class

The class the node would predict, usually its largest class.

Gini

A measure of class mixture. A pure node containing only one class has Gini impurity zero.

If a node has class proportions p_1, p_2, p_3 , its Gini impurity is

$$G = 1 - (p_1^2 + p_2^2 + p_3^2).$$

For a pure setosa node, the proportions are $(1, 0, 0)$, giving $G = 0$. For three equally represented classes, $G = 1 - 3(1/3)^2 = 2/3$. Learning tries to find splits that reduce impurity across the resulting groups.

The learned first question agrees with the visual separation noticed in the petal scatter plot. This connects exploration to the fitted model's behaviour.

11 Nearest neighbours and the model exercises

Material covered: Notebook Part 4.4 and Exercise 2.

11.1 How KNN makes a decision

K-Nearest Neighbours, abbreviated KNN, classifies an input by examining nearby training examples. The number k specifies how many neighbours to consider. With $k = 5$, it finds five nearby flowers and uses their labels to decide the output. With the default uniform weighting, each of those five neighbours has an equal vote.

For example, suppose three of the five neighbours are versicolor and two are virginica. The majority vote selects versicolor. Similarity is calculated using the numerical features. In this notebook, the default settings use Euclidean distance, the straight-line distance extended to the four-dimensional feature space.

For two features, Euclidean distance between examples a and b is

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}.$$

For Iris, the same calculation sums squared differences across all four measurements. You do not need to draw four dimensions to compute the distance.

11.2 The code and why it feels familiar

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print(f"KNN accuracy: {knn.score(X_test, y_test):.2f}")
```

The import brings in a new classifier. `n_neighbors=5` selects k . The same training and test arrays are used, which makes the example easy to compare with the tree. The checked KNN score is 1.00 on the original split.

During fitting, KNN retains information about the training examples for later searches. When a new input arrives, it finds nearby examples and combines their labels. There is no gradient-descent learning loop in this basic KNN classifier.

Scikit-learn's consistent method names help you reuse the general pattern. The details still depend on the estimator: some need preprocessing, some accept different inputs, and some use different scoring measures. The methods provide a shared interface rather than a guarantee that every model behaves identically.

11.3 Why scale and the neighbour count matter

Distance-based methods are affected by numerical scale. Suppose one feature ranges from zero to one and another ranges from zero to ten thousand. Differences in the second feature can dominate the distance calculation. Scaling can make feature contributions more suitable for the task. Even measurements with the same units can have different spreads.

The introductory Iris code uses the original measurements. Later experiments can compare appropriate preprocessing fitted only on training data. The good score in this small example does not remove the need to think about scale in other datasets.

A small k makes predictions depend strongly on a few nearby points. This can capture local detail and can also make predictions sensitive to noise. A larger k averages over a wider neighbourhood and can blur small groups. Suitable values depend on the data and should be selected using validation.

11.4 Exercise 2, task 1: change the test proportion

```
X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X, y, test_size=0.5, random_state=42
)
tree2 = DecisionTreeClassifier(random_state=42).fit(
    X_train2, y_train2
)
print(f"Tree accuracy with 50% test: "
      f"{tree2.score(X_test2, y_test2):.2f}")
```

With `test_size=0.5`, training uses 75 flowers and testing uses 75. The checked tree score is $68/75 \approx 0.9067$, which prints as 0.91. The original split gave 1.00.

Two things changed: the model received fewer training examples, and the set used for evaluation changed. Therefore, the difference cannot be attributed solely to training-set size. The example shows how experimental choices can affect a result.

The line creating `tree2` chains construction and fitting. `fit` returns the estimator itself, allowing this compact form. The two-step form used earlier is often easier to read while learning.

11.5 Exercise 2, task 2: vary the neighbour count

```
for k in [1, 5, 15]:
    knn_k = KNeighborsClassifier(n_neighbors=k).fit(
        X_train, y_train
    )
    print(f"KNN with k={k:2d}: accuracy "
          f"{knn_k.score(X_test, y_test):.2f}")
```

The loop fits three classifiers, one for each value of k . On the original 80/20 split, all three score 1.00. These results tie under this evaluation. They provide no basis for claiming that one neighbour count is universally best.

The solution cell uses `X_train` and `X_test`. If you edited the earlier cell to use a 50/50 split and reran it, those names now refer to different arrays. Reset the original 80/20 split before expecting these exact results. The separate variables ending in 2 let you explore the alternative split while preserving the originals.

Changing settings and checking their effects is useful for learning. In a serious experiment, use validation data for such choices and preserve an independent final test set.

11.6 Exercise 2, task 3: another new flower

```
print("Prediction:", iris.target_names[
    model.predict([[6.7, 3.0, 5.2, 2.3]])[0]
])
```

The original fitted tree predicts *virginica*. The petal length of 5.2 cm and width of 2.3 cm place this flower in the larger-petal region of the dataset. As before, the four values must follow the training feature order.

The variable `model` refers to the most recent classifier assigned to that name. If you have changed and rerun earlier cells, it may hold a different fitted tree. Reproducible results depend on both the code and its execution state.

12 Finding groups with K-Means

Material covered: Notebook Part 5.

12.1 What information the algorithm receives

The supervised classifiers received measurements and species labels during fitting. In this exercise, K-Means is supplied with the feature matrix **X**. It forms groups based on the four measurements. The known species labels are used afterward to help us interpret the grouping.

We choose three groups by setting `n_clusters=3`. The algorithm has not independently discovered that three is the correct number of species. We supplied that group count as an experimental choice.

12.2 The meaning of a centre

K-Means represents each cluster by a centre, also called a **centroid**. A centre has one coordinate for each feature. For Iris, it has four coordinates, corresponding to the four flower measurements.

A simplified description of the algorithm is:

1. Begin with initial centres.
2. Assign each example to its nearest centre.
3. Update each centre using the mean feature values of its assigned examples.
4. Repeat assignment and updating until a stopping condition is reached.

The objective is to reduce the sum of squared distances between examples and their assigned centres. This objective is often called within-cluster sum of squares or inertia. The result depends on the features, their scale, the chosen number of clusters, and initialisation.

12.3 Fit the clustering model

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X)
```

The constructor selects three clusters and a fixed random seed. `n_init=10` requests ten initialisations; the implementation keeps the run with the best objective value. This helps reduce dependence on one starting arrangement, although it does not guarantee a globally best solution.

`fit_predict(X)` fits the model and returns a cluster assignment for each of the 150 flowers. The resulting values are stored in a new column, `cluster`. This works because the rows of **X** and **df** still refer to the flowers in the same original order.

The call supplies no **y**. The presence of a species column elsewhere in **df** does not make it an input: the fitting call receives **X**, which contains only the four measurements. After adding the cluster column, **df** has seven columns if the earlier species column is still present.

12.4 Compare the two plots

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.scatterplot(data=df, x="petal length (cm)",
                y="petal width (cm)", hue="cluster",
                palette="Set2", s=60, ax=axes[0])
axes[0].set_title("Groups K-Means found (no labels used)")

sns.scatterplot(data=df, x="petal length (cm)",
                y="petal width (cm)", hue="species",
                s=60, ax=axes[1])
axes[1].set_title("The real species")
plt.tight_layout()
plt.show()
```

The left plot uses cluster assignments for colour, while the right plot uses species names. `ax=axes[0]` places the first scatter plot in the left panel, and `ax=axes[1]` places the second in the right panel. `palette="Set2"` selects a colour palette for the clusters.

Both panels display the same petal coordinates. K-Means used all four measurements even though only two are shown. Compare which points belong together, using the legends. A colour in one panel does not automatically correspond to the same species in the other panel.

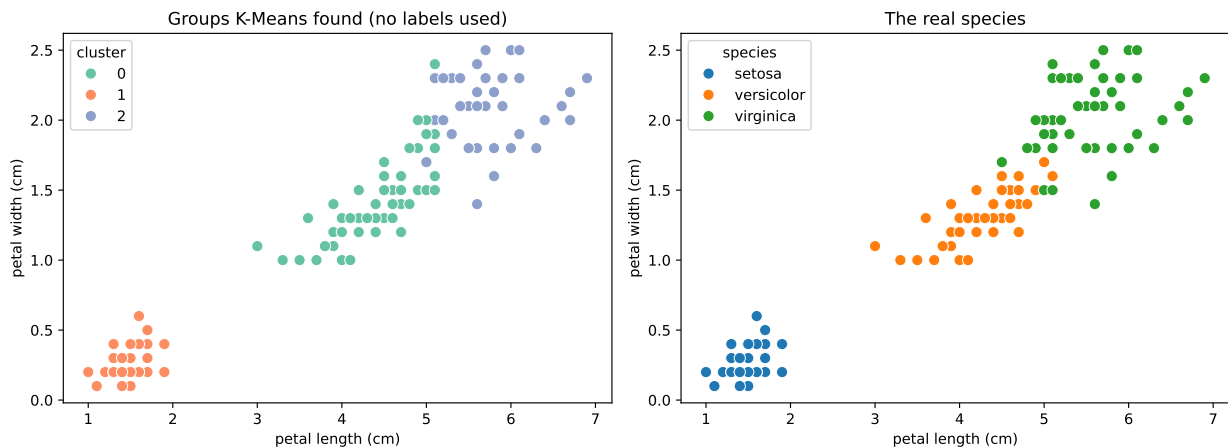


Figure 4: Cluster assignments and known species for the same flowers. Labels are used only for the comparison on the right.

12.5 What the checked grouping contains

Actual species	Cluster 0	Cluster 1	Cluster 2
setosa	0	50	0
versicolor	48	0	2
virginica	14	0	36

Cluster 1 contains all 50 setosa flowers. Cluster 0 contains most versicolor flowers and some virginica flowers. Cluster 2 contains most of the remaining virginica flowers and two versicolor flowers. The overlap is consistent with the less distinct separation seen in the petal plot.

The numbers 0, 1, and 2 are arbitrary identifiers. Reassigning their names leaves the grouping unchanged. Consequently, directly comparing `cluster == target` would give a misleading assessment: cluster 1 contains setosa, whose species code is 0.

If we associate each cluster with its majority species after inspecting the labels, $50 + 48 + 36 = 134$ of the 150 flowers match that association, about 89.3 percent. This is a descriptive, in-sample comparison after mapping cluster identities. It is not a held-out classification accuracy and should not be compared directly with the classifiers' 30-flower test scores.

K-Means seeks compact groups in the selected feature space. Species boundaries need not coincide perfectly with that objective. The exercise shows that meaningful structure can appear without supplying species labels to the fitting procedure.

13 Using ML responsibly and interpreting claims carefully

Material covered: Slide 27; responsible-use discussion in the speaker notes.

13.1 Representativeness and bias

A dataset can overrepresent some conditions and underrepresent others. If a model mainly sees clear images captured under bright lighting, its performance on dim images may be weaker. If a dataset contains measurements from only one machine type, transferring the model to another type needs evaluation.

Bias can arise from sampling, measurement, labelling, or the chosen objective. Historical labels may also reflect unfair decisions. A useful evaluation examines relevant groups and conditions separately when the data supports doing so. An average score can hide substantial differences across groups.

The face-analysis example in the speaker notes is intended to motivate this question: who and what are represented in the training and evaluation data? For any application, make that question specific to the population and circumstances of use.

13.2 Privacy and data handling

Data can contain personal or sensitive information, including identities, location histories, messages, or physiological measurements. Collect information appropriate to the task, establish permission to use it, and control access. Removing a name alone may leave other details that can identify a person.

Think about where the notebook, raw files, model inputs, and outputs are stored. Sharing a notebook can also share its visible outputs. Local inference may reduce transmission of raw data, but the surrounding application can still store or transmit information.

13.3 Data quality and confident mistakes

Wrong labels teach the model the wrong relationship. Incorrect units can produce plausible-looking values with the wrong meaning. A sensor problem can appear as a pattern that the model learns. Checking examples and measurement procedures is therefore part of model development.

Model outputs also need context. A classifier can select a class for an invalid input. A generated explanation can sound fluent while containing factual errors. Confidence-like scores require their own interpretation and validation. Check important outputs against reliable evidence and the model's intended scope.

13.4 Responsible learning habits

Keep a record of where your data came from, what you changed, and how you evaluated the result. When using generated code or explanations, run the code, inspect the result, and understand the steps before relying on them. In coursework, follow the applicable rules for attribution and assistance.

Describe your findings with their limits. “The tree classified all 30 flowers in this test split correctly” is a precise observation. Claims about deployment need additional evidence from the conditions in which the deployed system will operate.

14 Troubleshooting the notebook

Material covered: Notebook guide and common problems in the speaker notes.

14.1 An error is information about what happened

A Python error message usually includes a traceback: a record pointing to the statement where execution failed. Begin with the final line, which names the error type, then inspect the indicated code line. Fix the underlying problem and rerun the affected cells in order.

Symptom	Meaning and useful action
<code>NameError</code> for <code>temps</code> , <code>df</code> , or <code>model</code>	The name has not been defined in the current runtime. Run the earlier cell that creates it. If the runtime restarted, run from the top.
<code>IndentationError</code>	Python cannot interpret the block structure. Check the spaces after <code>for</code> , <code>if</code> , <code>else</code> , or <code>def</code> . Use consistent indentation.
<code>SyntaxError</code>	The statement does not follow Python’s syntax. Check quotes, brackets, commas, and colons near the indicated location.
<code>KeyError</code>	A dictionary key or table column is missing. Check spelling and inspect the available keys or columns.
<code>IndexError</code>	An index lies outside the collection’s available positions. A five-item list has indices 0 through 4.
<code>NotFittedError</code>	Prediction or evaluation was requested before fitting this estimator. Run the appropriate <code>fit</code> cell.
Expected a two-dimensional array	A single flower may have been supplied as a flat list. Use a collection containing one row, as in <code>[[5.0, 3.4, 1.5, 0.2]]</code> .
Different Exercise 1 average	Check whether <code>temps</code> contains five readings, the intended six readings, or extra appended values.
Different classifier accuracy	Check the split proportion, random seeds, model settings, library version, and which cells were rerun.
House-price loss becomes huge	Restore the intended learning rate and initial coefficient. Re-run the setup and learning loop together.
Notebook cannot be edited	If the shared notebook is view-only, create an editable copy in your own Drive.
Runtime disconnected	Reconnect. If the runtime was reset, rerun cells to recreate variables. Saving the notebook does not preserve the live runtime indefinitely.

14.2 A reliable way to recover your experiment

When the notebook state becomes confusing, restore the original settings, restart the runtime if necessary, and run the cells from top to bottom. Check the first error before continuing. A later error may simply be a consequence of an earlier cell failing.

Use separate variable names when comparing experiments. The provided solution uses `tree2` and arrays ending in 2 for the alternative split. This keeps the first model available and makes it easier to identify which result belongs to which experiment.

15 Review, worked answers, and preparation for Day 2

Material covered: Slides 28–29; notebook recap; speaker-note questions and homework.

15.1 Questions to answer in your own words

1. What does a trained model contain in the one-parameter house example?
2. For Iris, what are a sample, a feature, and the target?
3. Why does a target code of 2 still belong to a classification problem?
4. What distinguishes the guidance in supervised, self-supervised, and reinforcement learning?
5. Why are training accuracy and test accuracy different kinds of evidence?
6. Why are there two sets of brackets around a single new flower?
7. What does `random_state=42` help you control?
8. What does the error value in row 0 of the house-training table refer to?
9. Why can two clusters be renamed without changing the clustering result?
10. What should you check before using an ML output in a real application?

15.2 Worked answers

- 1. The learned coefficient.** The house model contains a learned w of approximately 0.50209 together with the chosen relationship $\hat{y} = wx$. Producing a prediction also requires knowing the input units and output units.
- 2. One flower, four measurements, one species.** A sample is a flower. Each measurement, such as petal length, is a feature. The species is the target. Its numeric and text forms represent the same target information.
- 3. The number is a category code.** Label 2 identifies *virginica*. Arithmetic size is not the meaning of that code. Classification concerns choosing the appropriate category.
- 4. Their learning signals.** Supervised learning uses known targets paired with inputs. Self-supervision constructs targets from existing data. Reinforcement learning uses rewards associated with actions and subsequent outcomes. A larger system can combine these settings.
- 5. The training examples influenced fitting.** A strong result on those examples can reflect close adaptation to them. A properly held-out test set provides evidence about performance on examples excluded from development, subject to the quality and representativeness of that test.

6. The model expects rows of samples. The inner list stores one flower's feature values. The outer list contains the collection of flowers. This gives the expected two-dimensional input structure.

7. Reproducibility of a random operation. A fixed seed helps repeat a split or a model's random choices under the same conditions. It does not select a guaranteed best model or guarantee identical outputs across all software versions.

8. The pre-update coefficient. The error 2582.33 was calculated using $w = 0.1$. The row prints after the update, so the displayed w is already approximately 0.3568.

9. Membership is the meaningful result. Cluster identifiers act as names. Replacing the name 0 with 2 everywhere, and adjusting other names consistently, leaves the same examples grouped together.

10. The intended use and supporting evidence. Check input validity, units, population or operating conditions, evaluation design, error types, and the action that follows a prediction. The appropriate checks depend on the application.

15.3 Questions beginners often ask

How much mathematics do I need? You can begin by running examples and learning what their operations mean. Averages, graphs, algebra, and basic probability are useful early foundations. Linear algebra and derivatives become increasingly useful as you study model behaviour and optimisation.

Does every ML task need a powerful laptop? The small examples here do not. Their datasets and models are modest. A browser-based runtime can also provide the computation. Larger experiments have different memory and compute requirements.

Will fitting always improve the result? A fitting procedure optimises a particular objective under particular settings. Unsuitable data, a poor representation, an unstable update, or an overly flexible model can produce weak results. The evaluation tells you how the chosen approach behaved.

Does the model keep learning during prediction? The fitted tree and KNN model in this notebook remain fixed while `predict` is called. Updating a deployed model requires a learning procedure designed or invoked for that purpose. Inference alone does not automatically retrain these models.

Should I learn TensorFlow or PyTorch first? The course introduces frameworks when neural networks become relevant. Understanding inputs, targets, losses, fitting, and evaluation transfers across frameworks. The choice for a project depends on the available code, deployment target, and team experience.

Can an ML tool replace engineering judgement? A useful prediction is one component of an engineered system. Measurement, design constraints, testing, interpretation, and responsibility for decisions remain part of the work. Learning these tools expands the tasks you can investigate and build.

15.4 Before Day 2

1. Run the notebook from a clean runtime, in order, and confirm that each cell completes.
2. Explain the input and output of every practical part in one or two sentences.
3. Complete the temperature-conversion and model exercises, then compare with the worked solutions.

4. Choose one application in your department and identify its inputs, target or objective, learning setting, and likely data source.
5. Practise lists, indexing, loops, conditions, and functions until you can modify the temperature example comfortably.
6. Bring one question about an output you found surprising.

Day 2 develops the data-handling work introduced here. You will spend more time with arrays, tables, plots, missing values, and the preparation required before a model can learn usefully.

16 Glossary and material guide

16.1 Day 1 glossary

Term	Meaning
Accuracy	Fraction of evaluated classifications whose predicted labels match the true labels.
Algorithm	A defined procedure for carrying out a task, such as fitting a decision tree.
Array	A numerical data structure used for organised calculations on collections of values.
Baseline	A simple reference result used to judge whether further modelling improves performance.
Cell	A notebook block containing code or explanatory text.
Centroid	The centre representing a cluster, often calculated as the mean of its members' feature values.
Classification	Prediction of a category.
Clustering	Grouping examples according to similarity under a chosen representation and objective.
DataFrame	A Pandas table with labelled columns and rows.
Dataset	A collection of examples used for analysis or learning.
Deployment	Integrating a fitted model into an application that can use it.
Dimension	One coordinate or feature in a numerical representation.
EDA	Exploratory data analysis: inspecting, summarising, and plotting data to understand it.
Epoch	One full pass over the training dataset in training procedures that use this terminology. The house loop evaluates all six examples in each step.
Feature	An input description supplied to a model.
Fit	Establish an estimator's learned state using supplied data.
Generalisation	Performance on further examples relevant to the intended application.
Gradient	Derivatives describing how a function changes as its parameters change.
Gradient descent	Repeatedly adjust parameters using the negative gradient of a loss.
Hyperparameter	A setting chosen outside the parameter-fitting process, such as KNN's neighbour count.
Inference	Producing outputs using a fitted model.
Label or target	The answer to be predicted in a supervised task.
Learning rate	A multiplier controlling the size of a gradient-based update.

Term	Meaning
Loss	A numerical objective measuring error or another training criterion.
Model	The representation used to map inputs to outputs or describe learned structure.
Overfitting	Learning training-specific detail that harms performance on new examples.
Parameter	A value established or adjusted during learning, such as the house model's coefficient.
Policy	A strategy for selecting actions in reinforcement learning.
Preprocessing	Operations that prepare raw data for a model.
Regression	Prediction of a numerical quantity.
Runtime	The active computing session in which notebook code executes.
Sample	One example in a dataset.
Seed	A value used to initialise a pseudo-random process for reproducibility.
Test set	Examples reserved for evaluating the final developed model.
Training set	Examples used to fit the model.
Validation	Evaluation used during model development and setting selection.

16.2 Where to find each part of the supplied material

Supplied material	Coverage in these notes
Slides 1–3	Getting started and the five-day roadmap.
Slides 4–9	ML foundations, history, computing, vocabulary, and the learning-loop chapter.
Slides 10–15	Five learning settings and worked identification questions.
Slide 16	The complete ML workflow.
Slides 17–20	Applications and embedded deployment concepts.
Slides 21–25	Python, tools, Colab, and Python essentials.
Slide 26	Flower classifier, evaluation, prediction, and tree interpretation.
Slides 27–29	Responsible use, recap, review, and preparation for Day 2.
Notebook Parts 0–1	Notebook operation, Python syntax, and Exercise 1.
Notebook Part 2	Iris loading, terminology, counts, and scatter plot.
Notebook Part 3	Full numerical learning-loop explanation and both experiments.
Notebook Part 4	Decision tree, prediction table, new flowers, tree plot, KNN, and Exercise 2.
Notebook Part 5	K-Means fitting, plots, and interpretation.
Speaker notes	Explanatory examples, discussion questions, exercise outputs, troubleshooting, and beginner questions integrated throughout.

16.3 Sources and verification notes

The principal sources are the three supplied teaching files:

1. *Day 1: Introduction to Machine Learning*, 29-slide PDF.
2. `Day1_Intro_to_ML.ipynb`, hands-on notebook including exercises and solutions.
3. `day1_speaker_notes.pdf`, 16-page speaker-note and notebook guide.

The notebook's non-empty code cells were executed in sequence to check the numerical results and generate the four figures included here. The checked environment used scikit-learn 1.8.0. Outputs

quoted in the notes refer to the original settings unless a changed setting is explicitly stated. Display formatting and some results can differ with software versions or modified notebook state.

Several brief teaching statements have been expanded for accuracy. Generative AI has a broader scope than a strict nested diagram suggests. Learning algorithms use different fitting procedures. Anomaly detection can use different learning settings. Embedded latency and resource use depend on the system. The training loop prints a pre-update loss with a post-update coefficient. Cluster identifiers require interpretation before comparing them with species codes.

The following official references were consulted for specific implementation details and resource limits. They are supplementary to the explanations developed from the supplied materials:

- Google Colab, FAQ: <https://research.google.com/colaboratory/faq.html>. Resource availability and runtime limitations.
- scikit-learn, *DecisionTreeClassifier*: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>. Estimator interface and score definition.
- scikit-learn, *KMeans*: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>. Initialisation and clustering interface.
- scikit-learn, *Common pitfalls and recommended practices*: https://scikit-learn.org/stable/common_pitfalls.html. Keeping learned preprocessing separate from test data.